ProjectPulse

Product & Technical Specification

Product: ProjectPulse — The Follow-Through Agent for Project Teams

Core promise: Make sure important commitments, requests, and decisions don't fall through the cracks.

Core concept: Evidence-to-Action Reconciliation

From conversations to verified follow-through.

---

1. The Problem

Things fall through the cracks because important commitments, requests, decisions, and updates are scattered across meetings, emails, and project-management platforms.

A team member might promise to deliver something during a meeting. Someone else might report that an earlier task has already been completed. A subsequent email might contain the requested deliverable. However, this information does not reliably make its way back into the relevant project records.

Project managers consequently spend time reconstructing what happened, identifying outstanding work, updating project records, and chasing people for follow-ups.

ProjectPulse addresses this gap by connecting new information from conversations to existing project records, resolving ambiguity, and determining whether subsequent evidence fulfills an outstanding commitment.

It is not simply a meeting summarizer or another task-management application. It is an intelligence and follow-through layer that connects conversations to existing work.

2. The Core Workflow

Consider a project team managing a hardware installation.

Step 1 — Capture a Source

A team meeting takes place. Someone says:

"I'll send the revised hardware specifications to Facilities by Friday."

The source may be an audio recording or an existing meeting transcript.

ProjectPulse should also support new information arriving through subsequent meetings, emails, or other sources.

Step 2 — Understand the Conversation

If an audio recording is provided, Meta's Muse Voice Transcribe converts it into text.

Alternatively, an existing transcript from Zoom or another meeting platform can be ingested directly. When both audio and a transcript are available, the transcript can provide a reference for verification.

Muse Spark analyzes the conversation and identifies relevant events, including:

- New commitments.
- Requests.
- Decisions.
- Progress updates.
- Completion claims.
- Blockers.
- Suggestions or proposed actions.

For each event, ProjectPulse should extract relevant information such as the responsible person, deliverable, project, deadline, and supporting source excerpt.

Not every statement should become a task. For example, "We could consider contacting Facilities" is a suggestion, not necessarily an agreed commitment.

Step 3 — Reconcile With Existing Project Records

ProjectPulse searches existing project records and its internal database for potentially related commitments and tasks.

It attempts to identify:

- The relevant project.
- The responsible person.
- Existing tasks that may represent the same commitment.
- Potential duplicates.
- Related commitments that are still outstanding.

When a matching task exists, ProjectPulse proposes linking the new commitment to that task rather than creating a duplicate.

When no matching task exists, it can propose a new task for human review.

Step 4 — Human Review

The user reviews proposed commitments and their relationships to existing records.

The user can confirm a task match, correct the owner or deadline, approve a new task, or reject an incorrect extraction.

Ambiguous suggestions and uncertain commitments should be sent for clarification rather than automatically becoming project tasks.

Step 5 — New Evidence and Reconciliation

This step is central to ProjectPulse.

New evidence does not necessarily have to arrive in a subsequent email. It can also appear during the original meeting.

For example, someone might say:

"I sent the revised specifications to Facilities yesterday. They have everything they need."

ProjectPulse should recognize this as a **completion claim**, not a new commitment. It searches for the related open task, identifies a potentially matching record, and attaches the statement as supporting evidence.

The same process applies when a subsequent email arrives with the specifications attached or when someone confirms completion during a later meeting.

Muse Spark evaluates how the new information relates to existing commitments. Possible interpretations include:

- **Potential completion:** Evidence suggests the commitment has been fulfilled.
- **Progress update:** Work has started or advanced, but is not necessarily complete.
- **Blocker:** Something is preventing completion.
- **Contradiction:** New information conflicts with an earlier commitment or status.
- **Unrelated information:** No relevant existing commitment is identified.

ProjectPulse must distinguish between someone saying they have completed a task and independently verified evidence of completion. It should not automatically resolve a commitment based only on the model's interpretation.

Step 6 — Close the Loop

When ProjectPulse identifies potential completion, it presents the original commitment, the existing project record, and the new supporting evidence.

A human reviews and approves the proposed resolution.

After approval, ProjectPulse updates the existing Notion task and records the supporting evidence and resolution history.

The task is then marked as resolved in ProjectPulse.

The complete cycle:

A team member makes a commitment → ProjectPulse identifies it → connects it to existing work → tracks it → recognizes new evidence → obtains human approval → updates the project record.

The system remembers what happened in one conversation, connects it to existing work, understands subsequent evidence, and helps the team follow through.

---

3. Product Interface

The MVP should have two primary sections.

A. Review Inbox

A centralized place for items that require human attention.

Examples:

- Uncertain commitment extraction.
- Ambiguous owners or deadlines.
- Potential matches to existing Notion tasks.
- Possible duplicate tasks.
- Conflicting information.
- Potentially completed commitments awaiting approval.
- Suggestions requiring clarification.

Each review item should show the relevant source excerpt and explain why ProjectPulse is requesting a decision.

B. Follow-Up Center

A centralized view of confirmed outstanding work across projects.

It should show:

- Confirmed outstanding commitments.
- Requests requiring action.
- Blockers.
- Items waiting on another person.
- Upcoming or overdue deadlines.
- Recently resolved commitments.

Clicking an item opens a detail panel containing:

- Original commitment or request.
- Supporting evidence and source information.
- Relevant project.
- Responsible person.
- Deadline, where available.
- Linked Notion task.
- Current status.
- Available actions.

The interface should make it easy to understand not only what needs attention, but also why it needs attention.

---

4. Technical Architecture

Core stack

Component	Technology	Responsibility
Backend	Python / FastAPI	API endpoints and application logic
Database	PostgreSQL	Open Loops, evidence, mappings, decisions, and state transitions
Workflow orchestration	LangGraph	Coordinate the extraction, retrieval, reconciliation, review, and execution workflow
Audio transcription	Meta Muse Voice Transcribe	Convert meeting recordings into text
Reasoning and extraction	Meta Muse Spark	Extract structured events and interpret new evidence

Project management integration	Notion API	Retrieve existing tasks and update approved records
Frontend	React / TypeScript	Review Inbox, Follow-Up Center, and item details

React is the preferred frontend for this MVP. The goal is to deliver a focused web application without splitting effort across multiple frontend implementations.

LangGraph Workflow

LangGraph should coordinate the application workflow.

The initial graph should contain modular nodes for:

1. **Source ingestion and normalization:** Accept audio recordings, meeting transcripts, and follow-up text.
2. **Transcription:** Call Muse Voice Transcribe when the source is audio.
3. **Event extraction:** Use Muse Spark to identify and structure relevant events.
4. **Context retrieval:** Retrieve relevant projects, people, existing tasks, and outstanding commitments.
5. **Entity and task matching:** Identify the most likely existing project records and detect potential duplicates.
6. **Evidence reconciliation:** Determine whether each new event represents a new commitment, completion claim, progress update, blocker, contradiction, or unrelated information.
7. **Human review:** Pause the workflow when a decision or clarification is needed.
8. **Action validation:** Validate the human-approved operation, target record, and current state.
9. **Execution:** Perform approved Notion updates or create approved tasks.
10. **Audit and persistence:** Store the resulting state transitions and evidence.

Use LangGraph's checkpointing and interrupt/resume capabilities for workflows that need to pause while waiting for human approval.

The graph should remain modular so additional sources and reconciliation capabilities can be introduced later.

PostgreSQL should remain the application's source of truth for commitments, evidence, and lifecycle history. LangGraph manages workflow execution; it should not replace the application's data model.

Important Engineering Areas

The engineering challenge is not simply extracting action items from transcripts. It is reliably connecting new information to existing work.

The implementation should pay particular attention to:

1. **Entity resolution:** Identifying the correct project, person, organization, and task.
2. **Existing-task matching:** Finding whether a new commitment relates to an existing task.
3. **Duplicate prevention:** Avoiding multiple project records for the same underlying commitment.
4. **Evidence provenance:** Retaining the original source, timestamp, speaker where available, and supporting excerpt.
5. **Commitment state transitions:** Distinguishing newly detected commitments, confirmed open items, progress updates, potential completions, and resolved items.
6. **Human approval:** Requiring a human decision when information or proposed actions are uncertain.
7. **Safe, idempotent updates:** Avoiding duplicate or unintended Notion updates when a workflow is retried.
8. **Temporal reasoning:** Resolving deadlines relative to the original source date and timezone, while distinguishing explicit dates from inferred dates.

Important Design Principle

The model interprets. The graph coordinates. The backend validates. The human resolves uncertainty. The system executes approved actions.

Muse Spark should not be responsible for directly changing project records or deciding that an action is authorized. The backend must validate the proposed operation before execution.

---

5. MVP Scope

The goal is to prove one complete evidence-to-action reconciliation workflow rather than build a broad project-management platform.

The initial demonstration should support:

1. An audio recording or pasted meeting transcript.
2. Extraction of a new commitment, a completion claim, and an ambiguous suggestion.
3. Retrieval of a small set of existing Notion projects and tasks.
4. Matching a new commitment to an existing task where appropriate.
5. Sending ambiguous information to the Review Inbox.
6. Recognizing a completion claim made during the meeting and matching it to an existing open task.
7. Ingesting one subsequent email or text update as another source of evidence.

8. Assessing whether that evidence indicates completion, progress, a blocker, or a contradiction.
9. Human approval of the proposed action.
10. Updating the relevant Notion record and displaying the resulting status in the Follow-Up Center.

For the MVP, use a small test Notion database and synthetic or otherwise authorized data.

Avoid building full integrations with Zoom, Gmail, Slack, or Google Drive. Support pasted transcripts and follow-up text initially; audio upload can exercise Muse Voice Transcribe. Broader integrations can come later.

Do not build a general-purpose chatbot, autonomous email-sending system, or a full project-management platform. The central feature is evidence-to-action reconciliation.

---

6. MVP Acceptance Criteria

The first version should demonstrate that:

- ProjectPulse can ingest a meeting recording or transcript.
- Muse Spark can identify commitments, requests, suggestions, progress updates, and completion claims.
- A completion claim made during a meeting can be matched to an existing open task.
- A new commitment can be linked to an existing task without creating a duplicate.
- An ambiguous suggestion is not automatically turned into a confirmed commitment.
- A progress update does not automatically resolve a task.
- New evidence from a subsequent source can be linked to an earlier commitment.
- Potential completion is presented for human approval.
- Notion is updated only after an approved action.
- Repeated execution does not create duplicate updates or records.
- The original evidence and resulting state changes remain traceable.

---

7. Implementation Priorities

Before building the full application, verify the critical integrations:

1. Confirm that Muse Spark can be called successfully and return structured extraction results.
2. Confirm that Muse Voice Transcribe can process a short test recording.
3. Connect to a test Notion database and successfully read and update a record.

4. Implement one end-to-end LangGraph workflow covering extraction, task matching, review, and an approved update.

These integration tests should come before extensive frontend work. If audio transcription is unavailable during development, use pasted transcripts as a fallback while retaining Muse Spark as the reasoning component.

---

8. Instructions for Claude Code

Treat this document as the product and technical specification for ProjectPulse.

Before writing application code:

1. Inspect the existing repository, its structure, installed dependencies, and current implementation.
2. Identify any existing components or code that can be reused.
3. Propose the application architecture, including the FastAPI backend, PostgreSQL data model, LangGraph workflow, Notion integration, and React frontend.
4. Propose the initial database schema for commitments, source evidence, projects, people, task mappings, human review decisions, and audit history.
5. Explain how the graph will handle both new commitments and completion claims found in the same meeting or in subsequent sources.
6. Identify integration risks, especially around Meta API access, Notion permissions, checkpointing, and safe updates.
7. Break implementation into small, testable milestones, beginning with the Muse and Notion integration spike.

Do not introduce unnecessary abstractions, agents, or integrations. Prefer a simple, modular implementation that can be expanded as the product evolves.

Do not start modifying or generating application code until you have presented the proposed implementation plan and I have reviewed it.

The immediate objective is to build a working vertical slice that demonstrates the complete follow-through cycle, not to build every planned feature.